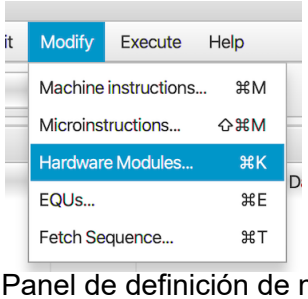
Guía Práctica 4 - *Arquitecturas microprogramadas - Tutorial CPU-Sim*

El principal objetivo de esta práctica es comenzar a construir un diseño de procesador propio muy simple pero que explore todos los conceptos básicos que mantiene cualquier microprocesador. Si bien cada uno puede luego definir un modelo con sus propias características, vamos a establecer algunas características comunes para poder guiarlos:

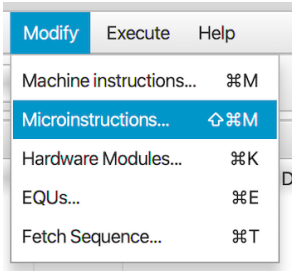
Para el procesador **P1** vamos a establecer su tamaño de ***arquitectura en 16bits***, es decir, sus instrucciones estarán expresadas en 16bits., estableciendo en principio que tendrá como máximo un ***SET de 16 instrucciones***, esto es, 4 de los 16bits serán destinados de establecer el identificador de operación (OPCode). El tamaño **máximo de memoria será de 1Kbyte**, esto es, direcciones de 10bits.

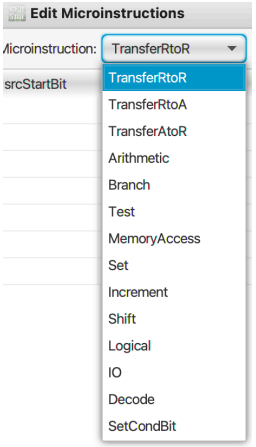
1. Ejecutar **CPUSim** y explore:
   - Panel de definición de Hardware.



-) Registers (Registros)
-) ConditionBit (Flags o banderas de estado)
-) RAM (memoria)

   - Panel de definición de microinstrucciones (instrucciones por hardware).





2. Defina el hardware (Registros y Memoria) que va a contener el procesador.
   - Program Counter “PC”, teniendo en cuenta para su tamaño que mantiene la **dirección de memoria** de la próxima instrucción a ejecutar.
   - Registro de Instrucción “IR”, teniendo en cuenta para su tamaño que cada instrucción puede ocupar como máximo 16 bits
   - Interfaz con la memoria (Registro de Datos “MDR” y Direcciones “MAR”), teniendo en cuenta que la cantidad máxima de direcciones es de 1Kbyte y que el tamaño de la arquitectura (para los datos inclusive) es de 16 bits.
   - Memoria RAM “Mem”, por ej. 128 direcciones de 8 bits c/u (reflexione cuántos bits serán necesarios para direccionarla).
   - Un registro de propósitos generales “A”, teniendo en cuenta que el tamaño de la arquitectura es de 16 bits.
   - Un registro de indicadores “FLAGS” que mantenga los siguientes indicadores: halt (fin de programa), carry (acarreo) y overflow (desbordamiento de representación).

- Definir el flag de “HALT” que indica que el proceso de un programa se debe interrumpir. Vamos a establecer que el bit 0 del registro de FLAGS indica del bit de HALT.

Edit Modules

Type of Module: ConditionBit

name	register	bit	halt
HALT	FLAGS	0	☑

3. Implementar (definir) las microinstrucciones para leer y escribir en memoria (a través de la interfaz). *Recuerde que definir microinstrucciones es implementar funcionalidades por hardware que luego se utilizan para definir el SET de instrucciones (ASSEMBLER) que brinda el procesador al usuario. Para las microinstrucciones podemos utilizar nombres mnemotécnicos para luego poder usarlas con facilidad.*
   - Implemente la microinstrucción de escritura en memoria “MDR->Mem[MAR]”

Type of Microinstruction: MemoryAccess

name	direction	memory	data	address
MDR->Mem[MAR]	write	Memoria	MDR	MAR

El nombre es sólo a efectos de expresar que lo que está almacenado en el registro de interfaz de datos con la memoria MDR se va a guardar en la memoria Mem en la dirección que indica el registro de direcciones de acceso a memoria MAR.

   - Implemente ahora la microinstrucción de lectura

4. Implementar (definir) las microinstrucciones para implementar la etapa inicial común a cualquier instrucción que es el FETCH. Para ello se necesita:
   - Conectar el PC con el MAR “PC->MAR”, es decir, que permita pasar la dirección de la próxima instrucción a ejecutar al MAR para luego leer la instrucción de la memoria memoria. Defina una microinstrucción de Transferencia de Registro a Registro que lo permita.
     Conectar el MDR con el IR “MDR->IR”, es decir, que permita pasar el valor leído de memoria (en este caso una instrucción) al registro de instrucción. Defina una microinstrucción de Transferencia de Registro a Registro que lo permita.
     En ambos casos tenga en cuenta la cantidad de bits establecidos, cuando manipulamos direcciones el tamaño es 10bits y cuando procesamos información almacenada en memoria (instrucciones o datos) 16bits que es el tamaño de la arquitectura.

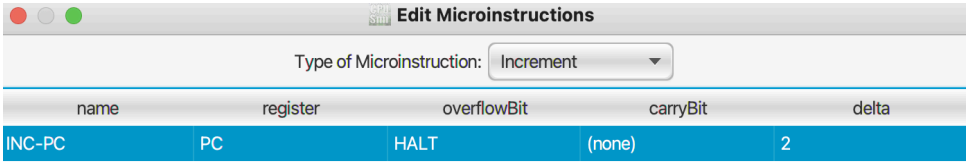
Edit Microinstructions

Type of Microinstruction: TransferRtoR

name	source	srcStartBit	dest	destStartBit	numBits
MDR->IR	MDR	0	IR	0	16
PC->MAR	PC	0	MAR	0	10

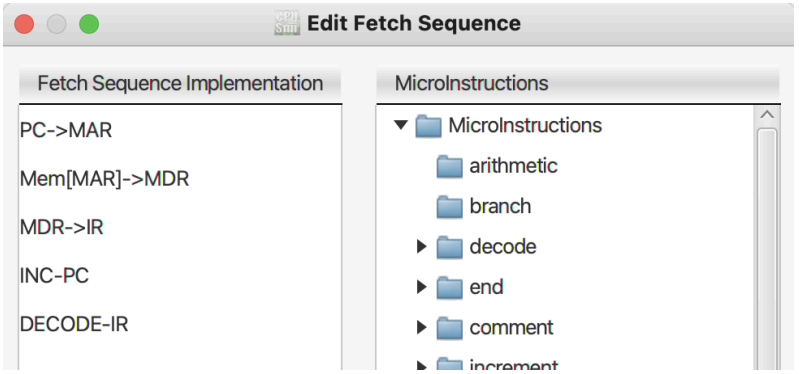
   - Definir una microinstrucción que incremente el PC para establecer el secuenciamiento “INC-PC”, es decir, que la próxima instrucción a ejecutar sea (por defecto) la siguiente a la que estamos procesando. Tenga en cuenta que cada instrucción ocupa 2 celdas de memoria ya que la arquitectura es de 16 bits. Además en caso de overflow, es decir que estamos intentando ejecutar una

instrucción que está fuera del rango de representación de direcciones, deberíamos interrumpir la ejecución, activando el HALT bit.



- Establecer la microinstrucción de “decodificación” que actúe sobre el registro de instrucción IR “DECODE-IR”.

5. Una vez implementado el hardware (microinstrucciones) necesarias, podemos establecer (microprogramar) la secuencia de implementación del **FETCH**:
   - Cargar la dirección de la próxima instrucción en el MAR
   - Leer de la memoria la instrucción
   - Pasar la instrucción al IR para su decodificación
   - Incrementar el PC para que la próxima instrucción sea la que sigue
   - Procesar la instrucción que se encuentra en el IR

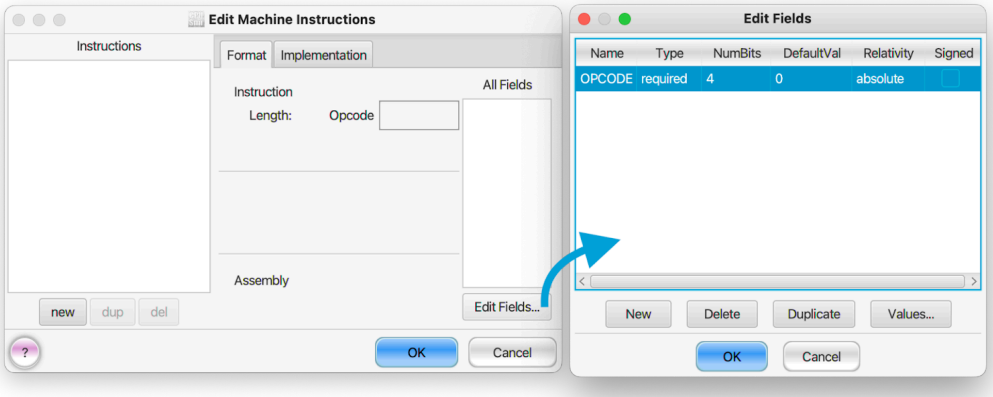


CONJUNTO DE INSTRUCCIONES (ISA)

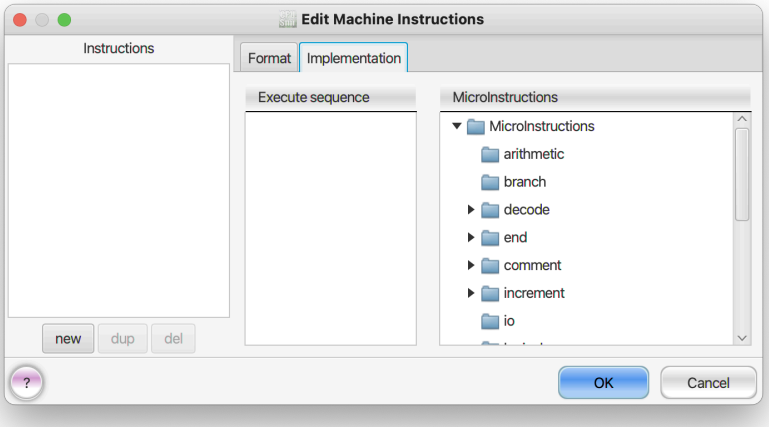
A partir de este estado, ya estamos en condiciones de comenzar a definir e implementar el conjunto de instrucciones (ASSEMBLY) que va a brindar nuestro procesador. Antes de comenzar la implementación de las instrucciones, exploramos el panel del simulador donde se definen e implementan las mismas. Al seleccionar en el menú la edición de Instrucciones, nos aparecerá el panel de edición que mostramos a continuación.

El panel presenta el listado de Instrucciones definidas, y para cada una de ellas, el **formato** y su **implementación**. El formato indica, de la cantidad de bits que tenemos para su definición, en nuestro caso 16bits, cuántos vamos a utilizar para el identificador de instrucciones (OPCODE) y cuántos para sus operandos. En este sentido y para facilitar la definición de los esquemas de las instrucciones podemos definir **campos (Fields)** con los cuales describiremos el formato de cada instrucción. *Cabe recordar que sólo es a fines de asistencia en la creación de nuestro procesador, ya que, como exploramos más adelante,*

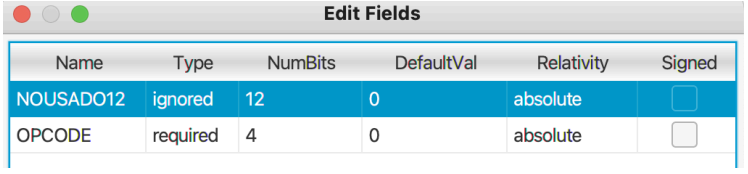
cuando se ejecutan los programas, tanto en los registros como en la memoria sólo hay ceros y unos.
Comencemos por definir un campo será común a todas las instrucciones que es la cantidad de bits que vamos a utilizar para identificarlas. Lo llamamos OPCODE y como acordamos que vamos a brindar como máximo 16 Instrucciones, lo vamos a establecer en 4 bits. En la medida que vayamos definiendo nuevas instrucciones iremos creando más campos, por ejemplo para los operandos esperados.



Una vez establecido el formato de la Inscripción, en la pestaña de implementación podremos **microprogramar** (indicar la secuencia) de microinstrucciones de hardware que la implementan.

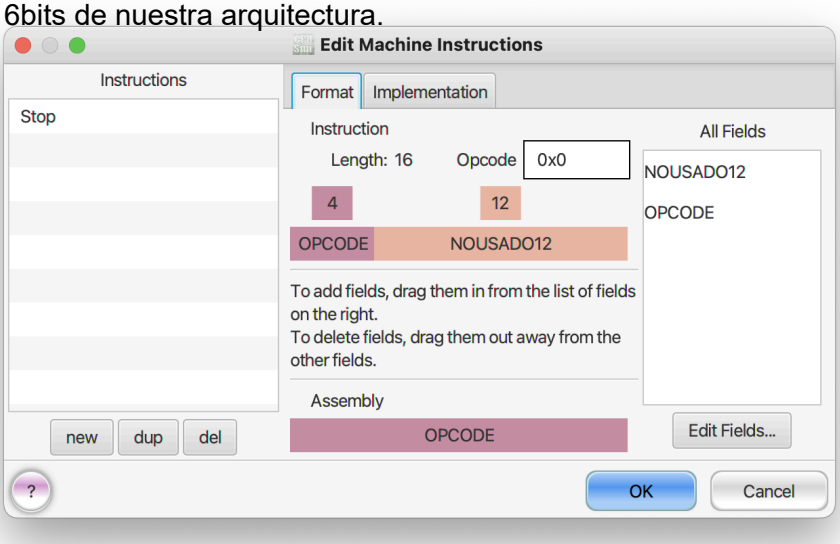


6. Comencemos con una introducción más que necesaria para indicar que el programa ha finalizado y que hay que **parar** la ejecución “Stop”.
Dado que esta instrucción NO necesita operando alguno, es simplemente indicar la finalización de un programa, necesitamos crear un campo para su definición que indique que el resto de los bits, luego de su identificación, no van a ser utilizados. Esto es necesario para establecer que cada instrucción que se cargue en memoria *complete la totalidad de sus bits*, sino podría darse el caso de que al cargar varias no *estén perfectamente alineadas* en 16bits.

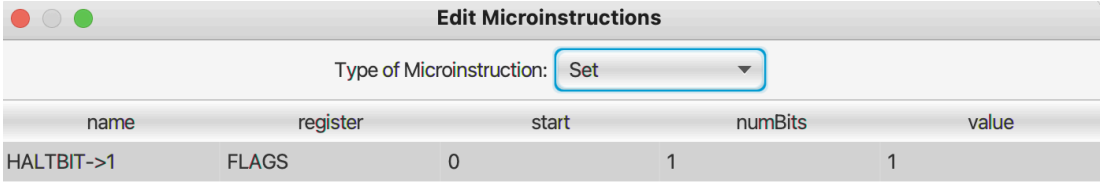


Se puede observar que tanto los campos OPCODE como el NOUSADO12 con sus respectivas cantidades de bits no es necesario considerar valores Enteros (**Signed**).

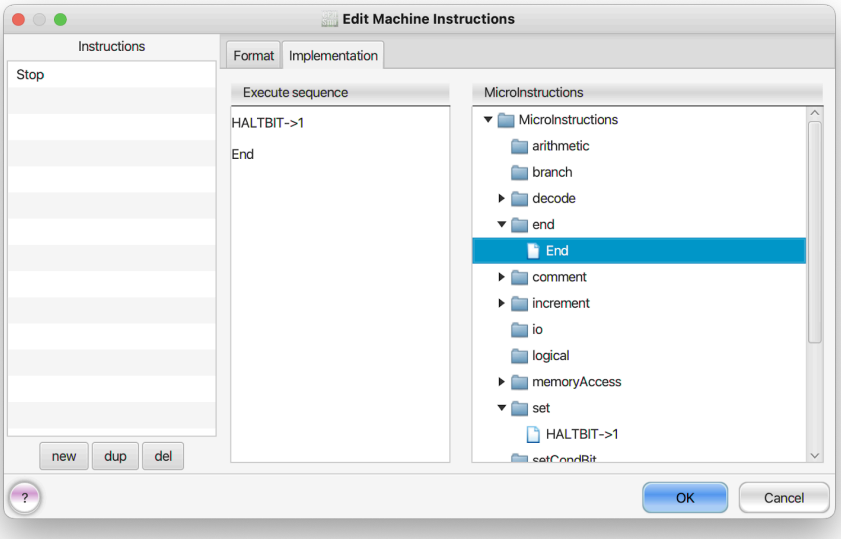
Además en caso de NOUSADO12 el tipo es **ignorado** porque solo lo consideramos para rellenar los 16bits de nuestra arquitectura.



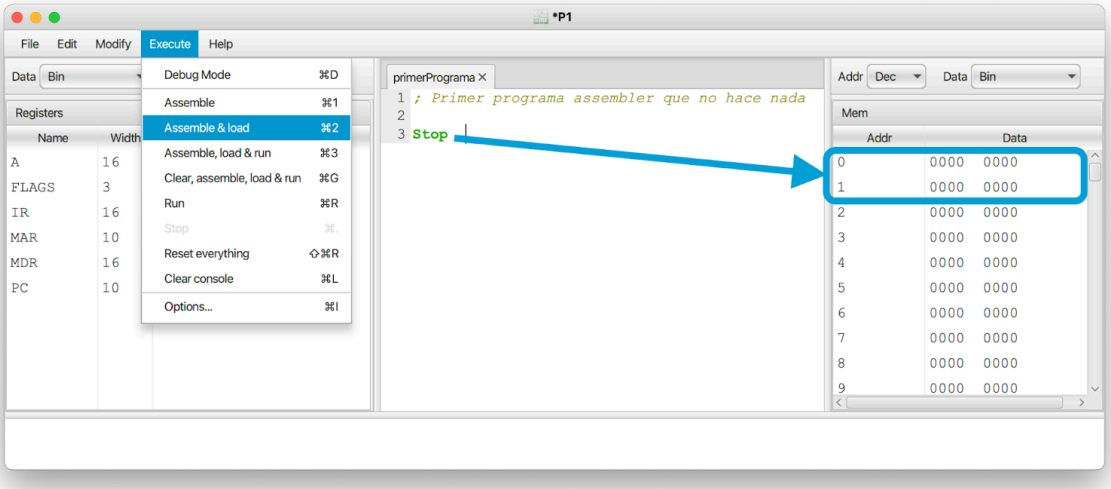
Una vez definido el formato, los primeros 4bits de identificación y el resto ignorado, ahora debemos microprogramar su comportamiento, es decir, qué microinstrucciones por hardware debe realizar durante su ejecución. Dado que simplemente debería encender (poner en 1) el bit de halt y que aún no tenemos una microinstrucción que lo haga, podemos crear la siguiente:



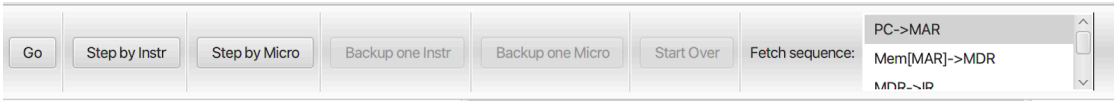
Ahora sí estamos en condiciones de poder implementar la primera instrucción “Stop” de nuestro ASSEMBLY, indicando que debe modificar el bit de finalización en 1. Es necesario indicar la finalización del microprograma con la microinstrucción por defecto que es “End” (que ya forma parte del simulador).



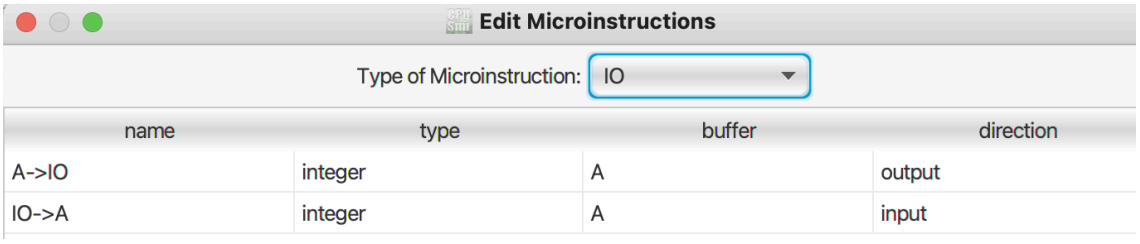
7. Ahora que ya tenemos una primera versión del procesador **P1** que la única instrucción disponible es “finalizar” :), podemos probarlo programando un primer programa que la use.



Comenzamos a escribir nuestro primer programa ASSEMBLY para nuestro procesador. El editor de programas, va a resaltar nuestra instrucciones y además soporta por ejemplo indicar comentarios (comenzando con “;”) que serán ignorados a la hora de compilar, cargar y ejecutar el programa. Una vez guardado el archivo del programa podemos compilar y cargar “Assemble & load”. Si bien como el OPCODE de la única instrucción es 0 y no tiene operandos, no lo notamos, pero en las dos primeras celdas de nuestra memoria está cargo el primer programa que sólo ejecuta stop.
Una vez compilado y cargado en memoria, podemos activar el “Debug mode” para poder ejecutarlo de manera controlada y siguiendo microinstrucción a microinstrucción, contemplando, una por una, tanto la secuencia común del FETCH como así también cada una de las microinstrucciones particulares de cada Instrucción.



8. De aquí en más iremos agregando instrucciones a nuestro procesador (y lenguaje assembly). Las próximas dos instrucciones serán una abstracción de Entrada/Salida (I/O), que nos permitan interactuar con la ejecución de un programa. Es decir, poder ingresar valores por teclado y mostrar valores por pantalla. Si bien a nivel de hardware requiere un análisis particular, en principio asumimos microinstrucciones que permiten leer y escribir valores entre Registros definidos y la I/O. Creamos las dos microinstrucciones que luego utilizaremos para implementar las correspondientes instrucciones de leer de teclado (**In**) y escribir por pantalla (**Out**) valores desde la I/O hacia el registro de propósitos generales A que tenemos definido.

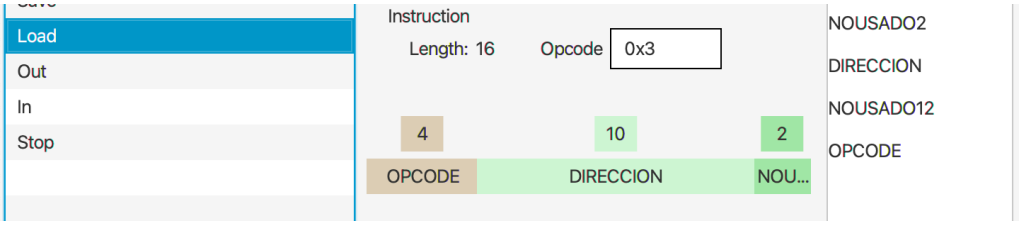


Utilizando estas microinstrucciones defina e implemente las instrucciones In y Out.
Pruebe el procesador con estas dos nuevas instrucciones ejecutando el siguiente programa:

```
In
Out
Stop
```

Prestar atención, luego de introducir un valor en la ejecución de la instrucción In, cómo queda almacenada la información en el registro A. Ingresar un valor negativo y observar qué representación utiliza. Nuevamente observe ahora cómo el código cargado en memoria ocupa 6 direcciones y que los OPCODE coinciden con los asignados a cada Instrucción.

9. Es hora de comenzar a interactuar con la memoria, esto es, poder leer y escribir valores en la memoria que tenemos en el registro de propósitos generales **A**. Antes de comenzar definir e implementar las correspondientes instrucciones “Load” y “Save”, debemos recordar y tener en cuenta que establecimos al comienzo que la capacidad máxima de memoria era de 1Kbyte de memoria, con lo cual, la cantidad de bits de direccionamiento es de diez bits. Debemos crear un campo “dirección” para poder establecer como operando de ambas Instrucciones. Además y dado que los 4 bits para el OPCODE más los 10 bits de la dirección es menor que el tamaño de nuestra arquitectura, debemos también crear un campo NOUSADO2 que nos rellene los dos bits que faltan.

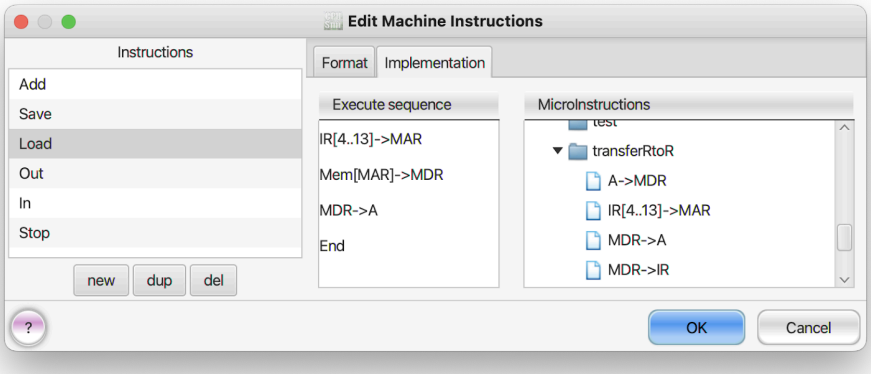


Para implementar esta Instrucción, debemos extraer la dirección desde el registro de instrucción que está en los últimos 10 bits [6..15], guardarla en el MAR (registro de direcciones) :

Type of Microinstruction: TransferRtoR

name	source	srcStartBit	dest	destStartBit	numBits
A->MDR	A	0	MDR	0	16
IR[4..13]->MAR	IR	4	MAR	0	10

Realizar una lectura de memoria y luego, una vez leído, pasar la información leída del MDR (registro de datos de memoria) al registro **A**. *(No olvide agregar **End** al final de cada implementación)*



De manera Dual, implemente “Save”, que tomando la dirección establecida como operando, almacena en la memoria en dicha dirección el valor actual del registro **A**.

10. De manera similar a la anterior, implemente ahora las operaciones aritméticas “Add” y “Sub” que toma como operando una dirección de memoria, y suma/resta lo que allí está con el valor actual del registro A y guarda el resultado en A. Ayuda, debería crear una microinstrucción que sume/reste los registros MDR y A, almacenando el resultado en A.

    Antes de probar nuestro procesador con estas últimas funcionalidades, es necesario incorporar el concepto de **etiqueta**. Una **etiqueta** es un nombre que identifica (en el código assembly) una dirección de memoria. Para declararlas simplemente escribimos un nombre, como si fuese una variable y luego “**:**”. Las **etiquetas no forman parte de la arquitectura**, simplemente son un artilugio del lenguaje de programación que *luego son reemplazadas durante la compilación en verdadera direcciones de memoria*. Las etiquetas pueden ser utilizadas tanto para indicar/reservar/asociar una dirección de memoria para **almacenar un valor** o para asociarla indicando una **instrucción del código** como veremos más adelante.
    Estas **etiquetas** las podemos utilizar como **operandos** que refieren a direcciones.

    Podemos probar las instrucciones con el siguiente programa que suma 3 valores ingresados:

ej10-secuenciaSumas ×

```
1  ;Programa que lee 3 valores y escribe
2  ;el resultado de la suma.
3  In        ;leo de consola un valor en A
4  Add tmp   ;sumo lo que tengo en A+tmp->A
5  Save tmp  ;guardo en tmp el resultado
6  In
7  Add tmp
8  Save tmp
9  In
10 Add tmp
11 Out      ;muestro por pantalla
12 Stop
13
14 tmp: .data 2 0; reserva de 2bytes para valor.
15
```

    Escriba, cargue y ejecute paso a paso (por microinstrucciones) el programa anterior, observando los cambios de estado en los registros y la memoria

11. Defina el Bit de Overflow (OF) y CarryOut (CO) en el registro de Flags en las posiciones 1 y 2 respectivamente. Modifique la Instrucción Add para que los actualice. Ejecute el programa anterior ingresando dos números suficientemente grandes para que individualmente sean representables con complemento a la base en 16 bits, pero cuya suma no. Observe si luego de realizar la instrucción de suma se enciende el bit OF.

Tomando el Control del PC

12. Implemente la instrucción “Jump” que salta a la instrucción que se encuentra en la dirección de memoria que indica su operando. Ayuda, al igual que para Load y Save, debemos obtener la dirección, pero ahora directamente copiarla en el PC para que la próxima instrucción a ejecutar sea la que se encuentra en dicha dirección.

    Para probar la funcionalidad podemos crear un programa que realiza un ciclo infinito solicitando números y realizando su sumatoria en A. Prestar atención cómo se utilizan las **etiquetas** para asociarlas a un punto en el programa, en este caso la etiqueta “**leerNum:**”, que luego, al compilar el programa es reemplazada por la dirección donde se carga la instrucción “**In**” de la línea 4.

NoTermina ×

```
1  ;Programa que utiliza Jump pero no termina
2
3  leerNum:
4      In        ;leo de consola un valor en A
5      Add tmp   ;sumo lo que tengo en A+tmp->A
6      Save tmp  ;guardo en tmp el resultado
7      Jump leerNum; salta a leer nuevamente
8      Out      ;muestro por pantalla
9      Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12
```

Hasta aquí hemos conseguido poder alterar el secuenciamiento predeterminado que realiza el FETCH, que es asumir que la próxima instrucción a ejecutar es la instrucción cargada en la siguiente posición en memoria, mediante modificar manualmente el PC con la dirección de la instrucción que necesitamos ejecutar. Si bien esto nos habilita a crear flujos de control propios, aún no resuelve la necesidad de poder alterar el flujo (siguiente instrucción) bajo alguna situación (condición) específica.

Antes que continuar con la implementación en términos de microinstrucciones de un *Salto Condicional*, debemos establecer qué es una condición en el contexto de una arquitectura (hardware). Lejos de la concepción de condiciones de lenguajes de alto nivel, como la que acompañan a sentencias condicionales o ciclos condicionales, las condiciones que podemos distinguir a nivel de hardware están definidas en términos de los estados de los registros de nuestra arquitectura. En mayor medida, pero no exclusivamente, el estado de los FLAGS. Por ejemplo, si el FLAG de **overflow** o el de **carry** están perdidos o apagados, o si el registro **A** tiene todos los bits en 0.

Para poder tomar diferentes alternativas en término del estado de algún registro o parte de él, podemos definir microinstrucciones (hardware) que, dependiendo de este estado, **omitan la ejecución** de una o más microinstrucciones siguientes, en la implementación de una instrucción.

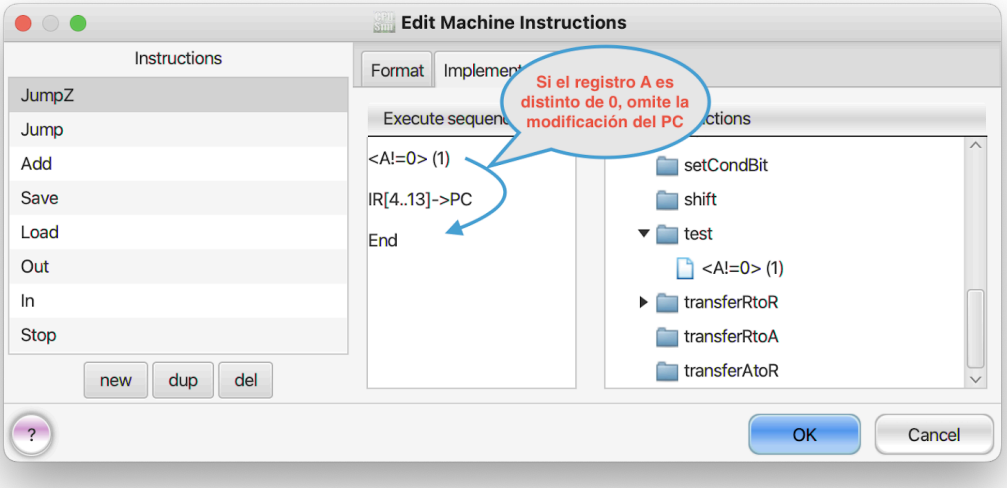
La siguiente pantalla muestra las opciones que tenemos en CPUSim para poder definir microinstrucciones denominadas de *test*. Como ejemplo definimos una microinstrucción que si el registro A, contemplando todos sus 16 bits [0..15], es **distinto (NE) a 0**, **debe omitir la siguiente microinstrucción en cualquier microprograma que la utilice**. Recordemos que el nombre de

cualquier microinstrucción es libre, pero recomendamos poner nombres acordes y siguiendo alguna lógica mnemotécnica. En este caso ponemos entre “< >” el estado a contemplar y entre “()“ la cantidad de microinstrucciones siguientes a omitir cuando esta situación se cumple: **<A=0>(1)**.

Type of Microinstruction: Test

name	register	start	numBits	comparison	value	omission
<A!=0> (1)	A	0	16	NE	0	1

13. Utilizando la definición anterior, implemente “JumpZ”, la instrucción que, al igual que “Jump”, toma como operando la dirección de la instrucción a saltar, pero ***sólo salta (cambia el PC) cuando A es igual a 0***.



14. Utilizando la instrucción anterior, construya un programa ASSEMBLY que lea valores por la entrada hasta que se ingrese un 0. Cuando lee un 0, muestra por pantalla el resultado de la suma acumulada y termina.